Assignment No. 1

Name : Ashutosh Sanjay More **PRN :** 124B1F036

Batch: A2 **Div :** A **Course :** Design and Analysis of Algorithms

Github Link : https://github.com/Ashutoshmore24/DAA_Submission/

---

1. Aim : To design and implement the Merge Sort algorithm using the Divide and Conquer strategy to efficiently arrange customer orders according to their timestamps, and to analyze its time and space complexity for large datasets.

2. Problem Statement : Design and implement a sorting algorithm using Merge Sort to efficiently arrange customer orders based on their timestamps.The solution should be capable of handling a large dataset of up to 1 million customer orders with minimal computational overhead.The performance of Merge Sort should also be analyzed in terms of:

- Best-case time complexity
- Average-case time complexity
- Worst-case time complexity
- Space complexity

Its performance should be compared with traditional sorting algorithms such as Bubble Sort, Insertion Sort, and Quick Sort.

3. Course Objectives :

3.1 : To understand the basics of computational complexity and the performance analysis of algorithms.

3.2 : To select appropriate algorithm design strategies for solving real-world computational problems.

3.3 : To understand and apply the Divide and Conquer paradigm.

3.4 : To implement an efficient sorting algorithm for handling large datasets.

4. Course Outcomes :

4.1 : Analyze the asymptotic performance of algorithms.

4.2 : Solve computational problems by applying suitable algorithmic paradigms such as Divide and Conquer and Greedy methodologies.

4.3 : Compare different algorithms based on their time and space requirements.

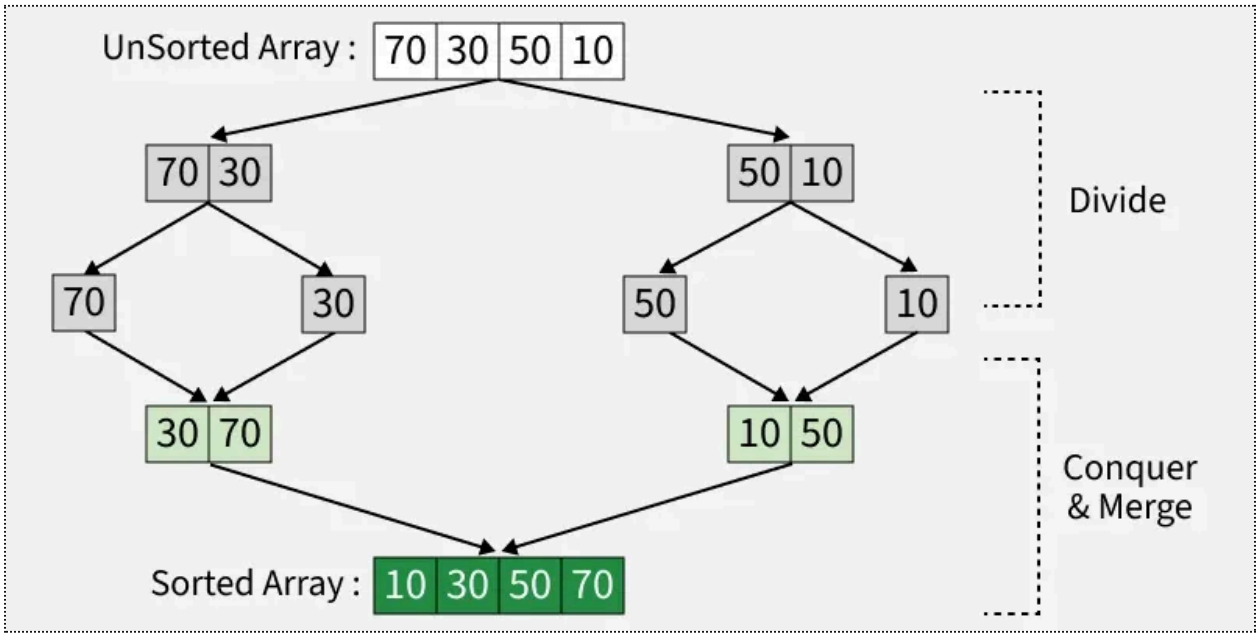
4.4 : Apply suitable algorithms to real-world problems involving large datasets.

Theory:

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the divide and conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Working of Merge Sort:

- **Divide:** Keep dividing the list or array into two equal halves again and again, until each part has only one element left.
- **Conquer:** Each sub array is sorted individually using the merge sort algorithm.
- **Merge:** The sorted sub arrays are merged back together in sorted order. The process continues until all elements from both sub arrays have been merged.



Complexity Analysis of Merge Sort:

Divide step: The array is recursively divided into two halves until each subarray has one element. Number of times we can divide an array of size $n = \log_2(n)$ (height of recursion tree).

Merge step: At each level of recursion, all n elements are merged.

```
Total work = number of levels × work per level = log₂(n) × n
= O(n log n).
```

Analysis of Merge Sort Time Complexity:

1. **Best Case Time Complexity :** Even if the array is already sorted, Merge Sort still **divides** and **merges** every element. Merging sorted sub arrays still requires scanning through all elements.

   Best Case = O(n log n)

2. **Average Case Time Complexity :** For randomly arranged data, the algorithm still divides into halves and merges them. On average, the number of comparisons during merging is proportional to n.

   Average Case = O(n log n)

3. **Worst Case Complexity :** Even if the array is sorted in **descending order** (worst arrangement for comparisons), Merge Sort still performs the same divide-and-merge procedure. Comparisons may be maximum, but still bounded by O(n log n).

   Worst Case = O(n log n)

Analysis of Merge Sort Space Complexity:

In merge sort, all elements are copied into an **auxiliary array** of size N, where N is the number of elements present in the unsorted array. Hence, the space complexity for **Merge Sort** is O(N).

Merge Sort Algorithm

Step 1: If it is only one element in the list, consider it already sorted, so return.
Step 2: Divide the list recursively into two halves until it can't no more be divided.
Step 3: Merge the smaller lists into new list in sorted order.

Pseudo code :

```
MERGE_SORT(array, left, right)

    if left >= right
        return

    mid = (left + right) / 2

    MERGE_SORT(array, left, mid)
    MERGE_SORT(array, mid + 1, right)

    MERGE(array, left, mid, right)
```

Merge Procedure

```
MERGE(array, left, mid, right)

    Create temporary array

    i = left
    j = mid + 1
    k = 0

    while i <= mid and j <= right

        if array[i] <= array[j]
            temp[k] = array[i]
            i = i + 1
        else
            temp[k] = array[j]
            j = j + 1

        k = k + 1

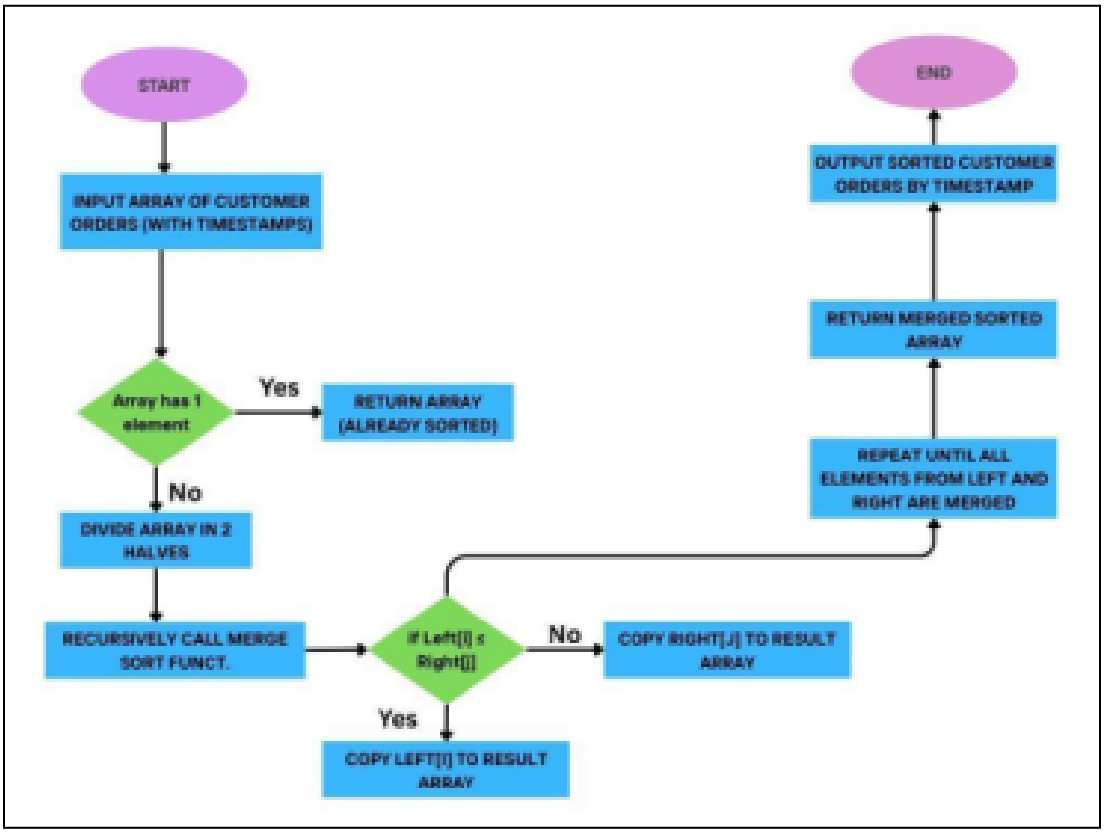
    while i <= mid
        temp[k] = array[i]
        i = i + 1
        k = k + 1

    while j <= right
        temp[k] = array[j]
        j = j + 1
        k = k + 1

    Copy temp into array[left...right]

END MERGE
```

5. Flow Diagram:



6. Comparison with Traditional Sorting Algorithms :

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)*	Usually No
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

7. Example:

Customer order timestamps:
[12:30, 09:15, 14:45, 10:00]

Step 1: Divide

- Split into halves → [12:30, 09:15] and [14:45, 10:00]
- Further split → [12:30], [09:15], [14:45], [10:00]

Step 2: Merge & Sort

- Merge [12:30] and [09:15] → [09:15, 12:30]
- Merge [14:45] and [10:00] → [10:00, 14:45] • Merge final two halves → [09:15,10:00, 12:30, 14:45]

Output:

Orders sorted by timestamp → **[09:15, 10:00, 12:30, 14:45]**

---

8. Implementation :

- **Github Code Link :**
- **Output link :**

9. Conclusion :

Merge Sort is a highly efficient and stable sorting algorithm that organizes customer orders based on their timestamps with **O(n log n)** complexity. Compared to traditional algorithms like Bubble Sort or Insertion Sort, which are too slow for large inputs, Merge Sort can easily handle up to **1 million records** without performance issues. Its ability to maintain the correct order and scalability makes it an excellent choice for real-world applications such as **e-commerce, order tracking, and scheduling systems**.
